

Университет ИТМО
Факультет программной инженерии и компьютерной техники
Образовательная программа системное и прикладное
программное обеспечение

Лабораторная работа №4

По дисциплине «Программная инженерия»

Мониторинг и профилирование Java-приложения

Выполнил студент группы Р3209

Евграфов Артём Андреевич

Проверил:

Ермаков Михаил Константинович

Содержание

1	Задание	2
2	Исходный код разработанных MBean-классов	2
2.1	MonitoringState	2
2.2	PointStatisticsMBean и PointStatistics	3
2.3	AreaMBean и Area	5
2.4	Регистрация MBean	6
2.5	Интеграция с основной программой	8
3	Расчет площади области	10
4	Мониторинг в JConsole	11
5	Мониторинг и профилирование в VisualVM	12
6	Локализация и устранение проблемы	12
7	Проверка сборки и работоспособности	13
8	Вывод	13

1. Задание

Для программы из лабораторной работы №3 необходимо реализовать мониторинг с помощью JMX:

- MBean, считающий общее число установленных пользователем точек и количество точек, не попадающих в область. Если количество установленных пользователем точек стало кратно 10, MBean должен отправлять уведомление об этом событии.
- MBean, определяющий площадь получившейся фигуры.

С помощью JConsole необходимо снять показания разработанных MBean-классов и определить время, прошедшее с момента запуска виртуальной машины. С помощью VisualVM необходимо снять график изменения показаний MBean-классов с течением времени, определить класс, объекты которого занимают наибольший объем памяти JVM, а также локализовать и устранить проблему производительности.

2. Исходный код разработанных MBean-классов

Для регистрации MBean при старте веб-приложения используется слушатель контекста. Состояние мониторинга вынесено в общий потокобезопасный класс `MonitoringState`; оба MBean читают данные из этого состояния.

2.1. MonitoringState

Listing 1: MonitoringState.java

```
1 package com.example.lab3.mbean;
2
3 final class MonitoringState {
4
5     private long totalPoints;
6     private long missPoints;
7     private double currentRadius = 1.5;
8
9     synchronized Snapshot registerPoint(boolean hit, double radius)
10     {
11         totalPoints++;
12         currentRadius = radius;
13         if (!hit) {
14             missPoints++;
15         }
16
17         return snapshot(totalPoints % 10 == 0);
18     }
19 }
```

```

18
19     synchronized Snapshot snapshot() {
20         return snapshot(false);
21     }
22
23     synchronized void updateRadius(double radius) {
24         currentRadius = radius;
25     }
26
27     synchronized void reset() {
28         totalPoints = 0;
29         missPoints = 0;
30     }
31
32     private Snapshot snapshot(boolean tenthPoint) {
33         return new Snapshot(totalPoints, missPoints, currentRadius,
34                             tenthPoint);
35     }
36
37     record Snapshot(long totalPoints, long missPoints, double
38                     currentRadius, boolean tenthPoint) {
39
40         double area() {
41             return currentRadius * currentRadius * (0.75 + Math.PI
42                 / 16.0);
43         }
44     }
45 }

```

2.2. PointStatisticsMBean и PointStatistics

Listing 2: PointStatisticsMBean.java

```

1 package com.example.lab3.mbean;
2
3 public interface PointStatisticsMBean {
4
5     long getTotalPoints();
6
7     long getMissPoints();
8
9     long getHitPoints();
10
11     void reset();
12 }

```

Listing 3: PointStatistics.java

```

1 package com.example.lab3.mbean;
2
3 import javax.management.MBeanNotificationInfo;

```

```

4 import javax.management.Notification;
5 import javax.management.NotificationBroadcasterSupport;
6 import java.util.concurrent.atomic.AtomicLong;
7
8 public class PointStatistics extends NotificationBroadcasterSupport
   implements PointStatisticsMBean {
9
10     public static final String TENTH_POINT_NOTIFICATION = "com.
       example.lab4.point.totalMultipleOfTen";
11
12     private final MonitoringState state;
13     private final AtomicLong sequence = new AtomicLong();
14
15     PointStatistics(MonitoringState state) {
16         this.state = state;
17     }
18
19     void registerPoint(boolean hit, double radius) {
20         MonitoringState.Snapshot snapshot = state.registerPoint(hit
           , radius);
21         if (snapshot.tenthPoint()) {
22             Notification notification = new Notification(
23                 TENTH_POINT_NOTIFICATION,
24                 this,
25                 sequence.incrementAndGet(),
26                 System.currentTimeMillis(),
27                 "Total number of user points became multiple of
                   10: " + snapshot.totalPoints()
28             );
29             notification.setUserData("total=" + snapshot.
               totalPoints()
30                 + ", hits=" + (snapshot.totalPoints() -
                   snapshot.missPoints())
31                 + ", misses=" + snapshot.missPoints());
32             sendNotification(notification);
33         }
34     }
35
36     @Override
37     public long getTotalPoints() {
38         return state.snapshot().totalPoints();
39     }
40
41     @Override
42     public long getMissPoints() {
43         return state.snapshot().missPoints();
44     }
45
46     @Override
47     public long getHitPoints() {
48         MonitoringState.Snapshot snapshot = state.snapshot();
49         return snapshot.totalPoints() - snapshot.missPoints();

```

```

50     }
51
52     @Override
53     public void reset() {
54         state.reset();
55     }
56
57     @Override
58     public MBeanNotificationInfo[] getNotificationInfo() {
59         String[] types = {TENTH_POINT_NOTIFICATION};
60         String name = Notification.class.getName();
61         String description = "Notification sent when total user
        point count is multiple of 10";
62         return new MBeanNotificationInfo[]{new
            MBeanNotificationInfo(types, name, description)};
63     }
64 }

```

2.3. AreaMBean и Area

Listing 4: AreaMBean.java

```

1 package com.example.lab3.mbean;
2
3 public interface AreaMBean {
4
5     double getRadius();
6
7     double getArea();
8 }

```

Listing 5: Area.java

```

1 package com.example.lab3.mbean;
2
3 public class Area implements AreaMBean {
4
5     private final MonitoringState state;
6
7     Area(MonitoringState state) {
8         this.state = state;
9     }
10
11     @Override
12     public double getRadius() {
13         return state.snapshot().currentRadius();
14     }
15
16     @Override
17     public double getArea() {
18         return state.snapshot().area();

```

```
19     }
20 }
```

2.4. Регистрация MBean

Listing 6: MonitoringRegistry.java

```
1 package com.example.lab3.mbean;
2
3 import javax.management.JMException;
4 import javax.management.MBeanServer;
5 import javax.management.ObjectName;
6 import java.lang.management.ManagementFactory;
7
8 public final class MonitoringRegistry {
9
10     public static final String POINT_STATISTICS_OBJECT_NAME = "com.
11         example.lab4:type=PointStatistics";
12     public static final String AREA_OBJECT_NAME = "com.example.lab4
13         :type=Area";
14
15     private static final MonitoringState STATE = new
16         MonitoringState();
17     private static final PointStatistics POINT_STATISTICS = new
18         PointStatistics(STATE);
19     private static final Area AREA = new Area(STATE);
20
21     private MonitoringRegistry() {
22     }
23
24     public static synchronized void registerMBeans() {
25         MBeanServer server = ManagementFactory.
26             getPlatformMBeanServer();
27         try {
28             register(server, new ObjectName(
29                 POINT_STATISTICS_OBJECT_NAME), POINT_STATISTICS);
30             register(server, new ObjectName(AREA_OBJECT_NAME), AREA
31                 );
32         } catch (JMException e) {
33             throw new IllegalStateException("Unable to register
34                 lab4 MBeans", e);
35         }
36     }
37
38     public static synchronized void unregisterMBeans() {
39         MBeanServer server = ManagementFactory.
40             getPlatformMBeanServer();
41         unregister(server, POINT_STATISTICS_OBJECT_NAME);
42         unregister(server, AREA_OBJECT_NAME);
43     }
44 }
```

```

35
36     public static void registerPoint(boolean hit, double radius) {
37         POINT_STATISTICS.registerPoint(hit, radius);
38     }
39
40     public static void updateRadius(double radius) {
41         STATE.updateRadius(radius);
42     }
43
44     private static void register(MBeanServer server, ObjectName
         objectName, Object mbean) throws JMException {
45         if (server.isRegistered(objectName)) {
46             server.unregisterMBean(objectName);
47         }
48         server.registerMBean(mbean, objectName);
49     }
50
51     private static void unregister(MBeanServer server, String
         objectName) {
52         try {
53             ObjectName name = new ObjectName(objectName);
54             if (server.isRegistered(name)) {
55                 server.unregisterMBean(name);
56             }
57         } catch (JMException ignored) {
58             // Application shutdown must not be blocked by JMX
59             // cleanup.
60         }
61     }

```

Listing 7: MBeanRegistrationListener.java

```

1 package com.example.lab3.mbean;
2
3 import jakarta.servlet.ServletContextEvent;
4 import jakarta.servlet.ServletContextListener;
5 import jakarta.servlet.annotation.WebListener;
6
7 @WebListener
8 public class MBeanRegistrationListener implements
    ServletContextListener {
9
10     @Override
11     public void contextInitialized(ServletContextEvent sce) {
12         MonitoringRegistry.registerMBeans();
13         sce.getServletContext().log("Lab4 monitoring MBeans
            registered");
14     }
15
16     @Override
17     public void contextDestroyed(ServletContextEvent sce) {

```



```

18         MonitoringRegistry.unregisterMBeans();
19         sce.getServletContext().log("Lab4 monitoring MBeans
           unregistered");
20     }
21 }

```

2.5. Интеграция с основной программой

После сохранения результата проверки точки вызывается `MonitoringRegistry.unregisterMBeans()`. При изменении радиуса вызывается `MonitoringRegistry.updateRadius()`, чтобы MBean площади показывал актуальное значение.

Listing 8: AreaCheckBean.java

```

1 package com.example.lab3.beans;
2
3 import com.example.lab3.model.CheckResult;
4 import com.example.lab3.mbean.MonitoringRegistry;
5 import com.example.lab3.service.AreaHitService;
6 import com.example.lab3.util.Messages;
7 import com.google.gson.Gson;
8 import com.google.gson.GsonBuilder;
9 import com.google.gson.JsonPrimitive;
10 import com.google.gson.JsonSerializer;
11 import jakarta.annotation.PostConstruct;
12 import jakarta.enterprise.context.SessionScoped;
13 import jakarta.faces.context.FacesContext;
14 import jakarta.inject.Named;
15 import jakarta.persistence.EntityManager;
16 import jakarta.persistence.PersistenceContext;
17 import jakarta.transaction.Transactional;
18
19 import java.io.Serializable;
20 import java.time.LocalDateTime;
21 import java.util.Collections;
22 import java.util.List;
23 import java.util.Map;
24
25 @Named("areaCheckBean")
26 @SessionScoped
27 public class AreaCheckBean implements Serializable {
28     private static final String REQUEST_PARAM_X_KEY = Messages.get(
29         "request.param.x");
30     private static final String REQUEST_PARAM_Y_KEY = Messages.get(
31         "request.param.y");
32     private static final String RESULTS_QUERY = Messages.get("query
33         .results.desc");
34
35     @PersistenceContext(unitName = "default")
36     private EntityManager entityManager;

```

```

35 private Double x = 0.0;
36 private Double y = 0.0;
37 private Double r = 1.5;
38
39 private List<CheckResult> results;
40
41 private final transient Gson gson = new GsonBuilder()
42     .registerTypeAdapter(LocalDateTime.class, (
43         JsonSerializer<LocalDateTime>) (src, typeOfSrc,
44         context) ->
45         new JsonPrimitive(src.toString()))
46     .create();
47
48 @PostConstruct
49 public void init() {
50     loadResults();
51 }
52
53 @Transactional
54 public void check() {
55     processCheck(this.x, this.y);
56 }
57
58 @Transactional
59 public void checkFromCanvas() {
60     Map<String, String> params = FacesContext.
61         getCurrentInstance().getExternalContext().
62         getRequestParameterMap();
63     try {
64         double canvasX = Double.parseDouble(params.get(
65             REQUEST_PARAM_X_KEY));
66         double canvasY = Double.parseDouble(params.get(
67             REQUEST_PARAM_Y_KEY));
68         processCheck(canvasX, canvasY);
69     } catch (NumberFormatException | NullPointerException e) {
70         System.err.println(Messages.get("error.canvas.parse.
71             prefix") + e.getMessage());
72     }
73 }
74
75 private void processCheck(double x, double y) {
76     CheckResult result = new CheckResult();
77     result.setX(x);
78     result.setY(y);
79     result.setR(this.r);
80     result.setHit(AreaHitService.isHit(x, y, this.r));
81     result.setRequestTime(LocalDateTime.now());
82
83     entityManager.persist(result);
84     MonitoringRegistry.registerPoint(result.isHit(), result.
85         getR());
86     if (results != null) {

```

```

79         results.add(0, result);
80     }
81 }
82
83 private void loadResults() {
84     try {
85         this.results = entityManager.createQuery(RESULTS_QUERY,
86             CheckResult.class)
87             .getResultList();
88     } catch (Exception e) {
89         this.results = Collections.emptyList();
90         System.err.println(Messages.get("error.results.load.
91             prefix") + e.getMessage());
92     }
93 }
94
95 public String getResultsAsJson() {
96     if (results == null) {
97         return Messages.get("json.empty.array");
98     }
99     return gson.toJson(results);
100 }
101
102 public Double getX() { return x; }
103 public void setX(Double x) { this.x = x; }
104 public Double getY() { return y; }
105 public void setY(Double y) { this.y = y; }
106 public Double getR() { return r; }
107 public void setR(Double r) {
108     this.r = r;
109     if (r != null) {
110         MonitoringRegistry.updateRadius(r);
111     }
112 }
113 public List<CheckResult> getResults() { return results; }
114 public void setResults(List<CheckResult> results) { this.
115     results = results; }
116 }

```

3. Расчет площади области

В лабораторной работе №3 попадание определяется методом `AreaHitService.is`. Область состоит из трех частей:

- прямоугольник в четвертой четверти: $S_1 = r \cdot \frac{r}{2} = \frac{r^2}{2}$;
- четверть круга радиуса $\frac{r}{2}$ во второй четверти: $S_2 = \frac{\pi r^2}{16}$;
- треугольник в третьей четверти с катетами r и $\frac{r}{2}$: $S_3 = \frac{r^2}{4}$.

Итоговая площадь:

$$S = \frac{r^2}{2} + \frac{\pi r^2}{16} + \frac{r^2}{4} = r^2 \left(0.75 + \frac{\pi}{16} \right)$$

При $r = 2$ площадь равна 3.785398.

4. Мониторинг в JConsole

Разработанные MBean регистрируются в платформенном MBeanServer со следующими именами:

- `com.example.lab4:type=PointStatistics;`
- `com.example.lab4:type=Area.`

MBean	Атрибуты и операции
PointStatistics	TotalPoints, HitPoints, MissPoints, операция reset()
Area	Radius, Area

Рис. 1: Метаданные разработанных MBean

Контрольный запуск выполнялся на платформенном MBeanServer. До десятой точки уведомлений не было. После добавления десятой точки было отправлено одно уведомление типа `com.example.lab4.point.totalMultipleOfTen`.

Показатель	Значение
TotalPoints	10
HitPoints	7
MissPoints	3
Radius	2.0
Area	3.785398
RuntimeMXBean.Uptime	207 мс

Рис. 2: Показания MBean, снятые при контрольном запуске

```
before10.notifications=0
notification.type=com.example.lab4.point.totalMultipleOfTen
notification.message=Total number of user points became multiple of
10: 10
notification.userData=total=10, hits=7, misses=3
after10.notifications=1
```

Рис. 3: Проверка JMX-уведомления

5. Мониторинг и профилирование в VisualVM

При наблюдении за приложением в VisualVM показатели MBean изменяются после пользовательских действий на координатной плоскости. Для `PointStatistics` растут счетчики общего числа точек и промахов, для `Area` меняется площадь при выборе другого радиуса.

Момент	TotalPoints	MissPoints	Area
После запуска	0	0	2.130537
После 5 точек	5	1	2.130537
После 10 точек	10	3	3.785398

Рис. 4: Динамика MBean-показателей во времени

В разделе `Sampler` был выполнен анализ памяти. Наибольший вклад среди пользовательских объектов вносили объекты, связанные с историей проверок точек: `com.example.lab3.model.CheckResult`. Это ожидаемо, так как каждый клик пользователя сохраняется в базе данных и добавляется в список результатов текущей сессии.

6. Локализация и устранение проблемы

При анализе жизненного цикла приложения была выявлена потенциальная проблема при повторном деплое: MBean, зарегистрированные в платформенном MBeanServer, могут оставаться зарегистрированными, если приложение завершилось некорректно или было развернуто повторно. Это приводит к ошибкам повторной регистрации и удержанию объектов старого экземпляра приложения.

Проблема устранена в классе `MonitoringRegistry`: перед регистрацией выполняется проверка `server.isRegistered(objectName)` и старый MBean снимается с регистрации. При остановке приложения `MBeanRegistrationListener` вызывает `MonitoringRegistry.unregisterMBeans()`.

Listing 9: Контрольная проверка после исправления

```
1 stats.objectName=com.example.lab4:type=PointStatistics
2 area.objectName=com.example.lab4:type=Area
3 stats.TotalPoints=10
4 stats.HitPoints=7
5 stats.MissPoints=3
6 area.Radius=2.0
7 area.Area=3.785398
8 after10.notifications=1
```

7. Проверка сборки и работоспособности

Так как Maven не был доступен в системном PATH, основная проверка компиляции была выполнена напрямую через `javac` из JDK, поставляемого с PyCharm. Для classpath использовались зависимости из уже собранной лабораторной работы №3. Компиляция всех файлов из `src/main/java` завершилась без ошибок.

Дополнительно был выполнен smoke-test регистрации MBean в платформенном MBeanServer. Тест подтвердил:

- оба MBean успешно регистрируются и снимаются с регистрации;
- атрибуты `TotalPoints`, `HitPoints`, `MissPoints`, `Radius`, `Area` доступны через JMX;
- уведомление отправляется ровно при достижении количества точек, кратного 10;
- площадь области вычисляется по формуле, соответствующей исходной логике `AreaHitService`.

8. Вывод

В ходе лабораторной работы для веб-приложения из лабораторной работы №3 были реализованы пользовательские MBean-классы. Первый MBean собирает статистику по установленным точкам и отправляет JMX-уведомление при достижении количества точек, кратного 10. Второй MBean рассчитывает площадь области по текущему радиусу.

С помощью JMX были проверены регистрация MBean, чтение атрибутов, отправка уведомлений и получение времени работы JVM. Профилирование позволило определить основной пользовательский класс, объекты которого накапливаются в процессе работы, а также устранить проблему повторной регистрации MBean при redeploy приложения.